

PayPal Agentic AI System

Technical Project Documentation

Version 2.0 / Submission v3 | 116-tool agentic API orchestration proof of concept

PayPal Agentic AI System – Technical Project Documentation

Version: 2.0 / Submission v3

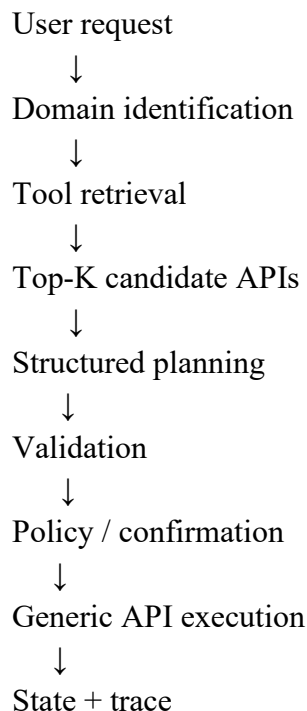
Purpose: Technical assignment / scalable agentic API orchestration proof of concept

1.Executive Summary

The PayPal Agentic AI System is an agentic API orchestration platform that allows users to interact with a large PayPal API catalogue using natural language.

The central engineering problem is tool selection at scale. A naive agent would expose every PayPal endpoint to the LLM. That approach becomes increasingly expensive, noisy, and difficult to control as the API catalogue grows.

This project instead builds a Tool Registry from a Postman collection and places a retrieval layer between the user request and the planner:



The current registry contains 116 sanitized PayPal API tools.

The system also separates three capabilities:

1. PayPal API operations — execute registered API tools.
2. RAG — answer knowledge/documentation questions.
3. System Search — discover available capabilities and inspect previous execution history.

This separation prevents a documentation question or system-search question from accidentally becoming an API execution request.

2.Problem Statement

A large API collection creates several engineering problems:

- The LLM should not receive hundreds or thousands of tool definitions at once.
- Similar endpoints can be difficult to distinguish from natural-language requests.
- Some business operations require multiple API calls.
- Later workflow steps depend on IDs returned by earlier steps.
- Sensitive operations require a confirmation boundary.
- API failures require controlled retries and authentication refresh.
- Execution history must survive process restarts.
- The system should remain understandable and observable during evaluation.

The project addresses these problems with retrieval, structured planning, state, validation, policy enforcement, and a generic executor.

3.Goals and Non-Goals

3.1 Goals

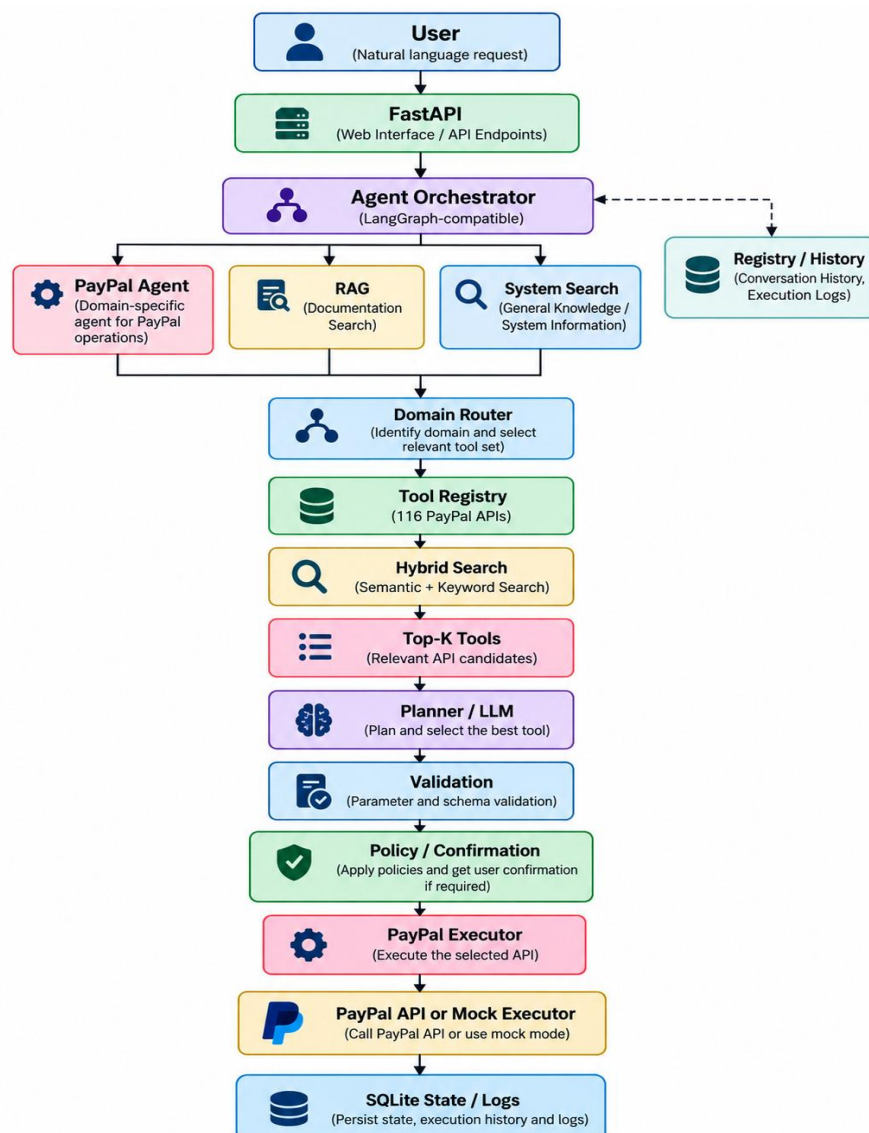
- Import a large Postman collection into a searchable registry.
- Support natural-language PayPal requests.
- Retrieve a bounded set of relevant tools.
- Support structured multi-step plans.
- Pass state between workflow steps.
- Validate path parameters and basic values before execution.
- Require confirmation for registered sensitive operations.
- Support mock execution for deterministic evaluation.
- Support real PayPal Sandbox execution through OAuth.
- Persist sessions and execution records.
- Provide an agent trace.

- Separate RAG and System Search from API execution.
- Provide a clear path from an MVP registry to thousands of APIs.

3.2 Non-goals

- Full production payment processing infrastructure.
- Distributed workflow orchestration in the MVP.
- Guaranteed correctness for arbitrary natural-language requests.
- A production-grade vector database/RAG stack.
- Complete PayPal business-domain coverage beyond the imported collection.
- Automatic execution of all possible sensitive operations without human approval.

4.High-Level Architecture



5.Component Responsibilities

5.1 FastAPI Gateway

``app/main.py``

Responsibilities:

- Expose HTTP endpoints.
- Validate incoming chat requests through Pydantic models.
- Route RAG/system-search questions separately.
- Forward PayPal operations to the orchestrator.
- Serve the demonstration UI.
- Expose health and execution-history endpoints.

5.2 Agent Orchestrator

``app/agent/orchestrator.py``

Responsibilities:

- Load persisted session state.
- Route the request into the PayPal agent path.
- Infer a domain.
- Retrieve candidate tools.
- Request a structured plan.
- Validate each step.
- Apply policy/confirmation checks.
- Execute tools sequentially.
- Promote IDs into workflow state.
- Persist results.
- Produce an execution trace.

The orchestrator is the main stateful workflow controller.

5.3 LangGraph Integration

``app/agent/graph.py``

LangGraph is used as the orchestration abstraction around the agent entry point. The graph delegates execution to the orchestrator workflow.

The current MVP intentionally keeps the graph small. The business logic remains in a testable Python orchestrator rather than hiding all behavior inside a large graph definition.

A production evolution could split the graph into explicit nodes such as:

route → retrieve → plan → validate → policy → execute → persist → respond

5.4 Tool Registry

``data/tool_registry.json``

The registry is generated from the Postman collection using ``scripts/import_postman.py``.

Each tool contains metadata such as:

- ``tool_id``
- ``name``
- ``service``
- ``domain``
- ``group``
- HTTP method
- endpoint path
- description
- path parameters
- query parameters
- required parameters
- request example
- request schema
- risk classification
- confirmation requirement
- searchable text

The registry currently contains ****116 tools****.

5.5 Retrieval Layer

``app/retrieval/retriever.py``

The current retrieval implementation combines:

- TF-IDF lexical/semantic similarity
- Tool-name word overlap
- Domain matching
- HTTP-method/write-intent signal

The result is a ranked Top-K candidate list.

Example:

User request:

"Send an invoice for \$50 to john@example.com"

Registry:

116 tools



Invoice domain filter/boost



Ranked candidates



Top 8 candidates



Planner

The retrieval interface is intentionally isolated so a production backend can replace TF-IDF without changing the planner/executor contract.

6.Planning

`app/agent/planner.py`

The planner has two modes.

6.1 LLM mode

Enabled when `OPENAI\_API\_KEY` is configured.

The planner receives:

- User query
- Retrieved candidate tools only
- Current workflow state

The LLM is instructed to return structured JSON containing:

```
{
  "intent": "...",
  "domain": "...",
  "steps": [],
  "clarification": null
}
```

The planner is explicitly constrained to registered candidate tool IDs.

6.2 Deterministic fallback mode

When no LLM key is configured, the fallback planner supports reproducible evaluation workflows including:

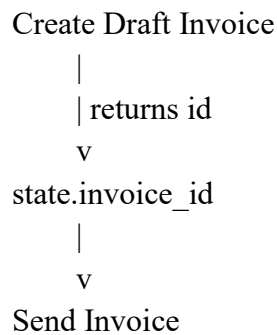
- Create and send invoice
- Create and capture order
- Show order details
- Show invoice details
- Missing-parameter clarification

This makes the demo independent of an external LLM service.

7. Multi-Step Workflow State

A multi-step API operation may need to pass an identifier from one tool to another.

Example:



The planner can express a later dependency using:

```
{{state.invoice_id}}
```

The orchestrator resolves the reference before execution.

Supported state promotion includes common identifiers such as:

- `order\_id`
- `invoice\_id`
- `payment\_id`
- `capture\_id`
- `refund\_id`
- `dispute\_id`
- `subscription\_id`

8. Validation

Validation occurs before tool execution.

Current checks include:

- Required path parameters are present.
- Numeric amount values are valid.
- Amounts must be greater than zero.
- Path placeholders must be resolvable.

If information is missing, the system returns a clarification response instead of executing an incomplete request.

Example:

User:

Show me order details

Agent:

Please provide the order ID.

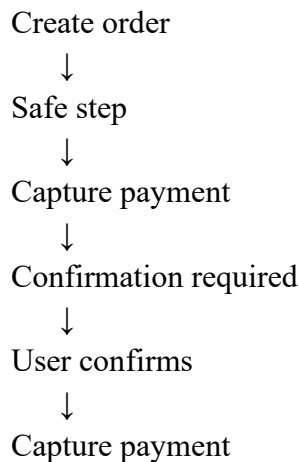
9. Policy And Confirmation

`app/policy/engine.py`

Tools in the registry can be marked as requiring confirmation based on risk classification.

The policy layer prevents sensitive operations from being executed when confirmation is required but not supplied.

Example:



This is a deliberately simple MVP policy boundary. Production systems should implement authorization, user identity, role-based permissions, transaction limits, audit requirements, and stronger policy evaluation.

10.PayPal Executor

``app/paypal/executor.py``

The executor is responsible for converting a registered tool definition into an HTTP request.

Responsibilities include:

1. Resolve path parameters.
2. Resolve state references.
3. Construct query parameters.
4. Construct request body.
5. Add required HTTP headers.
6. Obtain an OAuth token.
7. Execute the HTTP request.
8. Add PayPal request IDs for mutating calls.
9. Refresh the token once after an initial 401.
10. Retry bounded 429/5xx/network/timeout failures.
11. Normalize the result into an execution record.

The LLM never receives or manipulates the OAuth client secret directly.

11.OAuth

``app/paypal/oauth.py``

The OAuth manager obtains PayPal access tokens using configured client credentials.

Production recommendations:

- Store credentials in a managed secret store.
- Never log tokens.
- Rotate credentials periodically.
- Use separate Sandbox and Production credentials.
- Restrict network access where appropriate.

12.Mock Execution

Mock mode is enabled by default:

PAYPAL MOCK_MODE=true

The mock executor returns deterministic synthetic results for the key demonstration workflows.

Examples:

Create Draft Invoice
→ MOCK-INVOICE-1001

Create Order
→ MOCK-ORDER-1001

Mock mode is important because it makes the evaluator workflow:

clone → install → run → demo

independent of PayPal credentials.

13.RAG Capability

`app/tools/rag.py`

RAG is intentionally separate from PayPal execution.

The current MVP uses the local `data/knowledge/` directory and lexical retrieval to select relevant content.

Example:

How does PayPal invoicing work?
↓
RAG search
↓
Knowledge passages
↓
Answer + source names

The current implementation is a lightweight local RAG demonstration. A production implementation can add embeddings, a vector store, document chunk metadata, reranking, and source-level evaluation.

14.System Search

`app/tools/system\_search.py`

System Search is designed for questions about the system itself rather than PayPal execution.

Examples:

What APIs are available for invoices?

and:

What did my last request do?

The second use case relies on persisted execution records.

15.State and Persistence

``app/state/store.py``

SQLite stores:

Sessions

- Session ID
- Serialized state
- Updated timestamp

Executions

- Execution ID
- Session ID
- Tool ID
- Status
- Result JSON
- Creation timestamp

The persisted state allows a confirmation workflow to resume after the first request.

For production, replace SQLite with a shared persistent data layer such as PostgreSQL and use Redis or another low-latency store where appropriate.

16.Observability and Traceability

The application exposes a structured trace with stages including:

request
routing
retrieval
planning
clarification
validation
policy
execution
completed

Each execution also has an execution ID.

The design allows future integration with:

- OpenTelemetry
- LangSmith
- centralized logs
- metrics
- distributed tracing

Production logs should include correlation IDs while excluding secrets and sensitive payment data.

17.Security Model

Security controls implemented in the MVP include:

- Environment-based credentials
- Sanitized tool registry
- Mock mode by default
- Registered-tool-only planning
- Parameter validation
- Confirmation gates
- OAuth token handling in executor layer
- One-time 401 refresh
- Bounded retries
- PayPal request IDs for mutating operations

Security improvements for production are documented in `SECURITY.md`.

18.Error Handling

The executor distinguishes common classes of failure:

- Authentication failure
- HTTP 4xx failure
- HTTP 429 rate limiting
- HTTP 5xx service failure
- Timeout
- Network failure
- Missing parameters
- Unknown tool
- Unresolved path parameters

Retryable failures use bounded exponential backoff.

A production system should additionally define domain-specific retry policies and idempotency behavior for every mutating endpoint.

19.API Contract

POST `/chat`

Request:

```
{
  "session_id": "demo-1",
  "message": "Send an invoice for $50 to john@example.com",
  "confirmed": false
}
```

Response structure:

```
{
  "session_id": "demo-1",
  "answer": "Invoice MOCK-INVOICE-1001 was created and sent successfully.",
  "status": "completed",
  "tool_used": "...",
  "execution_id": "...",
  "plan": [],
  "trace": []
}
```

The complete endpoint reference is in `API\_REFERENCE.md`.

20.Testing Strategy

Tests are organized around system boundaries rather than only individual functions.

Registry tests

Verify that the expected 116 tools are present.

Retrieval tests

Verify that invoice requests retrieve invoice-related tools, including nested groups.

Planning tests

Verify multi-step invoice planning and clarification behavior.

Workflow tests

Verify invoice execution and order confirmation/resumption.

Policy tests

Verify sensitive operations are gated.

State tests

Verify persistence and history.

Run:

```
pytest -q
```

21.Evaluation Metrics

A production evaluation set should contain natural-language requests with expected tool IDs and workflow outcomes.

Recommended metrics:

Metric | Meaning

Domain accuracy | Correct PayPal domain identified

Top-1 accuracy | Correct tool ranked first

Top-K recall | Correct tool present in candidate set

Plan exact match | Correct workflow sequence

Clarification accuracy | Correctly asks for missing information

Safety block rate | Sensitive actions stopped correctly

Execution success | API calls complete successfully

p50/p95 latency | Runtime performance

Retry rate | Frequency of transient failures

Trace completeness | Required stages represented

22.Scaling Strategy

Stage 1 — 50 APIs

Use:

- SQLite
- TF-IDF
- metadata filtering
- simple deterministic/LLM planning

Stage 2 — 500 APIs

Introduce:

- stronger domain indexes
- hybrid BM25 + vector retrieval

- caching
- improved reranking
- evaluation datasets

Stage 3 — 5,000+ APIs

Introduce:

- hierarchical capability taxonomy
- service/domain/operation indexes
- distributed retrieval
- OpenSearch/pgvector/Qdrant
- dedicated reranker
- Redis/PostgreSQL
- durable workflows
- centralized observability

The planner still receives only a small final candidate set.

23. Framework Trade-offs

LangGraph

Best fit for explicit stateful workflows and controlled branching.

LangChain

Strong integration ecosystem, but the project does not need to depend on its abstractions for every layer.

LlamaIndex

Strong RAG/document tooling; less central to the API execution orchestration problem.

CrewAI

Useful for role-based multi-agent patterns, but introduces more agent complexity than required for this MVP.

DSPy

Useful for prompt/program optimization and evaluation, rather than being the primary runtime workflow engine.

Custom orchestration

Maximum control, but requires more custom state and workflow code.

The chosen architecture therefore uses LangGraph-compatible orchestration, a custom tool registry/retrieval layer, FastAPI, and a generic PayPal executor.

24.Demo Acceptance Criteria

An evaluator should be able to verify the following in a few minutes:

1. Application starts locally.
2. `/health`` reports the registry count.
3. UI is reachable.
4. Invoice workflow completes in mock mode.
5. Order workflow pauses for confirmation.
6. Agent trace shows routing/retrieval/planning/validation/execution.
7. RAG answers a documentation question.
8. System Search discovers APIs or history.
9. Tests pass.
10. README provides a repeatable setup path.

25.Future Production Roadmap

Phase 1 — Current evaluation build

- 116-tool registry
- mock workflows
- LLM/fallback planning
- local retrieval
- SQLite state
- trace
- tests

Phase 2 — Production retrieval

- embeddings
- vector database
- BM25 hybrid retrieval
- reranker

- retrieval evaluation dataset

Phase 3 — Production orchestration

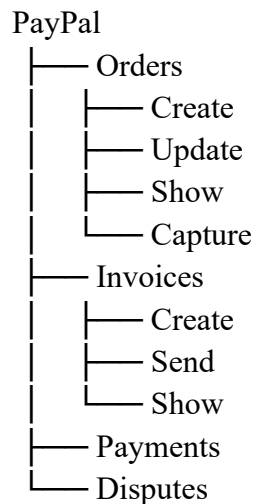
- durable workflow engine
- distributed workers
- PostgreSQL/Redis
- stronger policy engine
- role-based authorization
- approval service

Phase 4 — Production observability

- OpenTelemetry traces
- metrics dashboards
- alerting
- audit logs
- redaction

Phase 5 — Capability graph

Represent capabilities hierarchically:



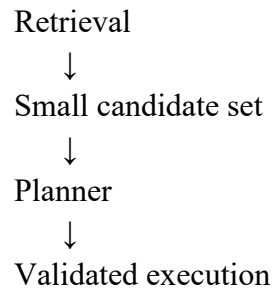
This can make very large registries easier to navigate and evaluate.

26. Conclusion

The project demonstrates a scalable pattern for turning a large API collection into an agent-operable capability system without placing the entire API catalogue in the LLM context.

Its most important architectural property is **bounded tool context**:

Large API registry
↓



That pattern allows the same architecture to grow from a small API collection toward hundreds or thousands of capabilities while retaining explicit control over planning, execution, state, safety, and observability.